

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО
ITMO University

Лабораторная работа 3

По дисциплине Объектно-ориентированное программирование

Тема работы Использование выражений

Обучающийся Сакулин Иван Михайлович

Факультет Факультет прикладной информатики

Группа К3221

Направление подготовки 11.03.02 Инфокоммуникационные технологии и
системы связи

Образовательная программа Программирование в инфокоммуникацион-
ных системах

Обучающийся

(дата)

(подпись)

Сакулин И.М.

(Ф.И.О.)

Руководитель

(дата)

(подпись)

Иванов С.Е.

(Ф.И.О.)

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ	3
1 УПРАЖНЕНИЕ 1.....	4
1.1 Задание 1	4
1.2 Задание 2	5
1.3 Задание 3	6
2 УПРАЖНЕНИЕ 2.....	8
2.1 Задание 1	8
2.2 Задание 2	11
2.3 Задание 3	12
2.4 Код-ревью.....	15
ЗАКЛЮЧЕНИЕ	17

ВВЕДЕНИЕ

Отчёт по выполнению лабораторной работы. В тексте работы представлены результаты выполнения двух упражнений про условные операторы и циклы, содержащие решения нескольких заданий. Для каждого задания предоставлены рисунки с кодом или выводом консоли. Для последнего задания реализованы основной и дополненный код.

Цель - изучение и приобретение навыков использования управляющих конструкций для организации вычислений.

1 УПРАЖНЕНИЕ 1

1.1 Задание 1

В проекте «Shapeifelse» была реализована и протестирована программа (рисунок 1).

```
using System;

namespace Shapeifelse
{
    0 references
    internal class Program
    {
        0 references
        static void Main(string[] args)
        {
            Console.Write("x=");
            float x = float.Parse(Console.ReadLine());
            Console.Write("y=");
            float y = float.Parse(Console.ReadLine());
            if (x * x + y * y < 9 && y > 0)
                Console.WriteLine("внутри");
            else if (x * x + y * y > 9 || y < 0)
                Console.WriteLine("вне");
            else
                Console.WriteLine("на границе");
        }
    }
}
```

Рисунок 1 — Проверка попадания точки в область

1.2 Задание 2

В проекте «Calc_switch» была реализована и протестирована программа (рисунок 2), добавлена обработка ошибок некорректного пользовательского ввода.

При делении на ноль происходит исключение «DivideByZeroException», при неправильном символе операции, результат не рассчитывается.

```
using System;

namespace Calc_switch
{
    0 references
    internal class Program
    {
        0 references
        static void Main(string[] args)
        {
            try
            {
                Console.Write("A = ");
                double a = double.Parse(Console.ReadLine());
                Console.Write("OP = ");
                char op = char.Parse(Console.ReadLine());
                Console.Write("B = ");
                double b = double.Parse(Console.ReadLine());

                bool ok = true;
                double res = 0;

                switch (op)
                {
                    case '+': res = a + b; break;
                    case '-': res = a - b; break;
                    case '*': res = a * b; break;
                    case '/': res = a / b; break;
                    default: ok = false; break;
                }

                if (ok)
                    Console.WriteLine("{0} {1} {2} = {3}", a, op, b, res);
                else
                    Console.WriteLine("Операция не определена");
            }
            catch (DivideByZeroException)
            {
                Console.WriteLine("Деление на ноль не определено");
            }
            catch (FormatException)
            {
                Console.WriteLine("Некорректный ввод");
            }
        }
    }
}
```

Рисунок 2 — Применение оператора switch

1.3 Задание 3

В проекте «YaerChecker» была реализована (рисунок 3) и протестирована (рисунок 4) программа, определяющая является ли год високосным, предусмотрены ошибки пользовательского ввода (рисунок 5).

```
using System;

namespace YearChecher
{
    0 references
    internal class Program
    {
        0 references
        static void Main(string[] args)
        {
            try
            {
                Console.Write("Enter year number: ");
                int year = int.Parse(Console.ReadLine());

                if (year <= 0)
                {
                    Console.WriteLine("Positive integer!");
                    return;
                }

                if (year % 400 == 0 || (year % 4 == 0 && year % 100 != 0))
                    Console.WriteLine("YES");
                else
                    Console.WriteLine("NO");
            }
            catch (FormatException)
            {
                Console.WriteLine("Input must be a positive integer");
            }
        }
    }
}
```

Рисунок 3 — Определение високосного года (код)

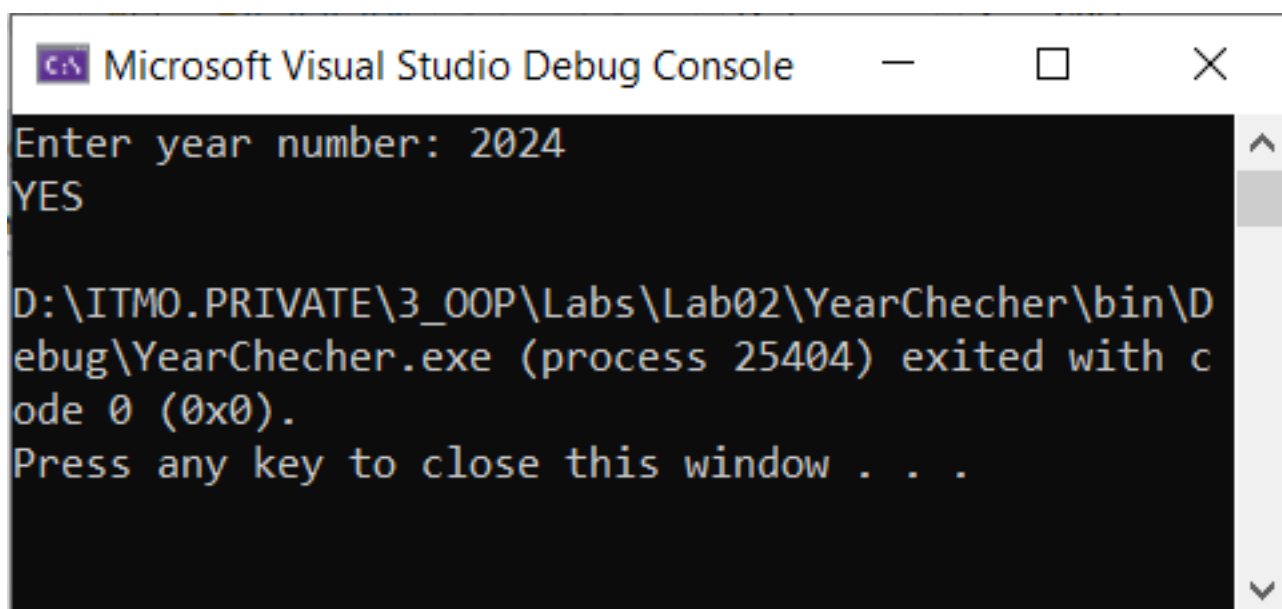


Рисунок 4 — Определение високосного года (правильный ввод)

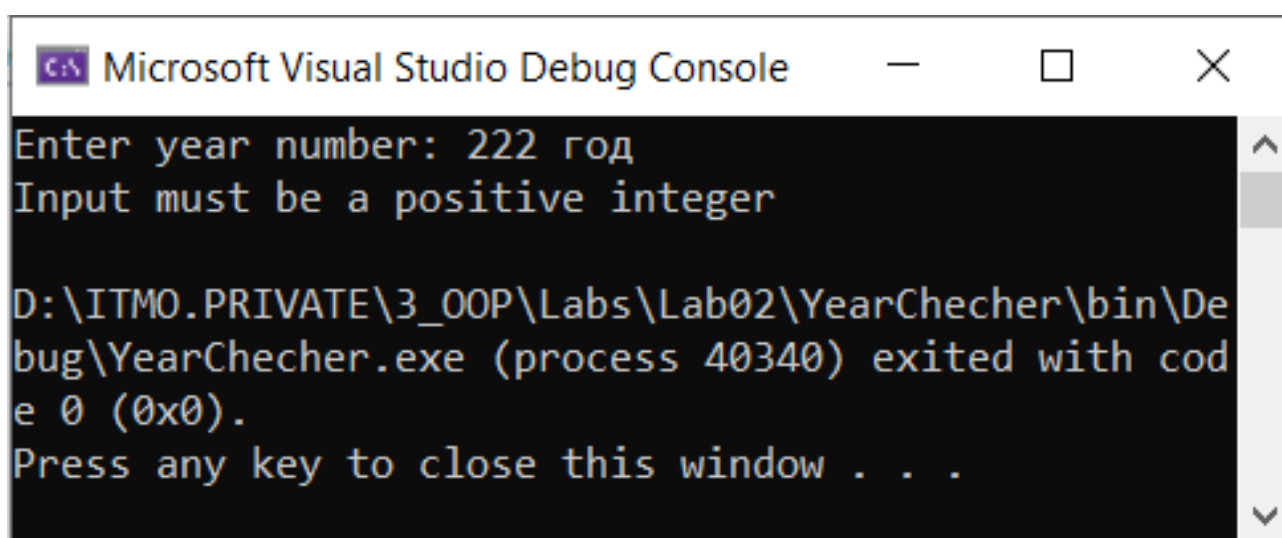
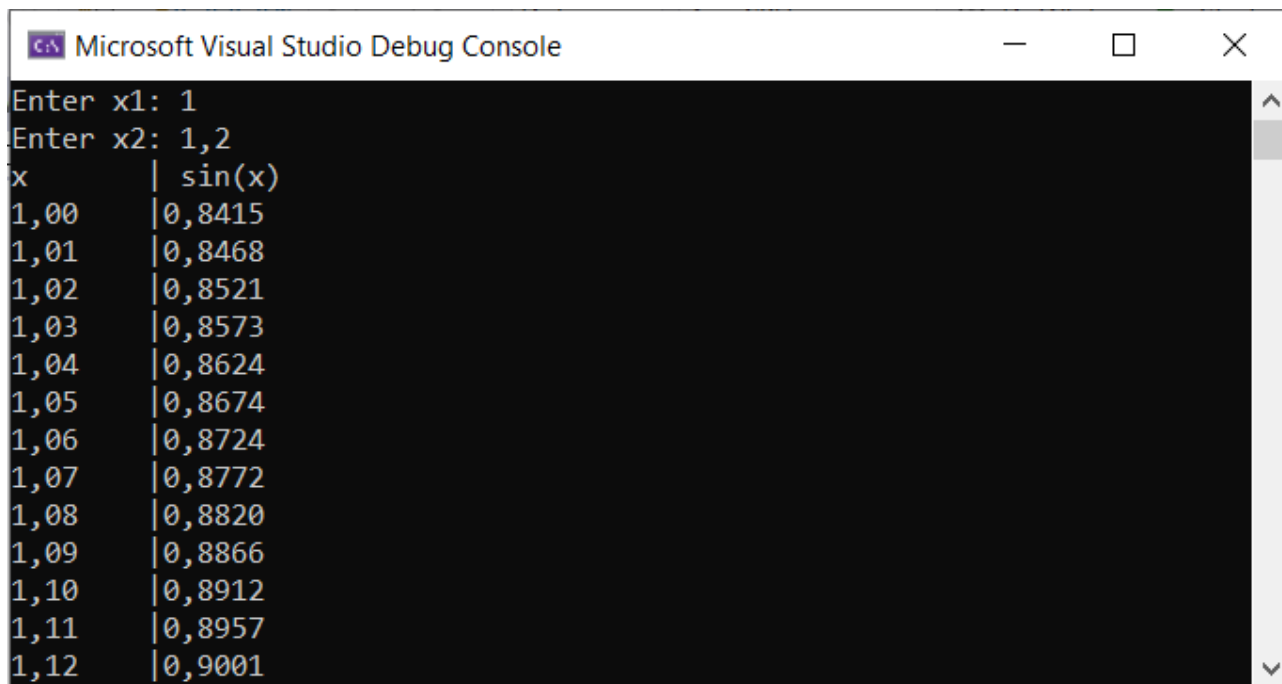


Рисунок 5 — Определение високосного года (ошибка пользователя)

2 УПРАЖНЕНИЕ 2

2.1 Задание 1

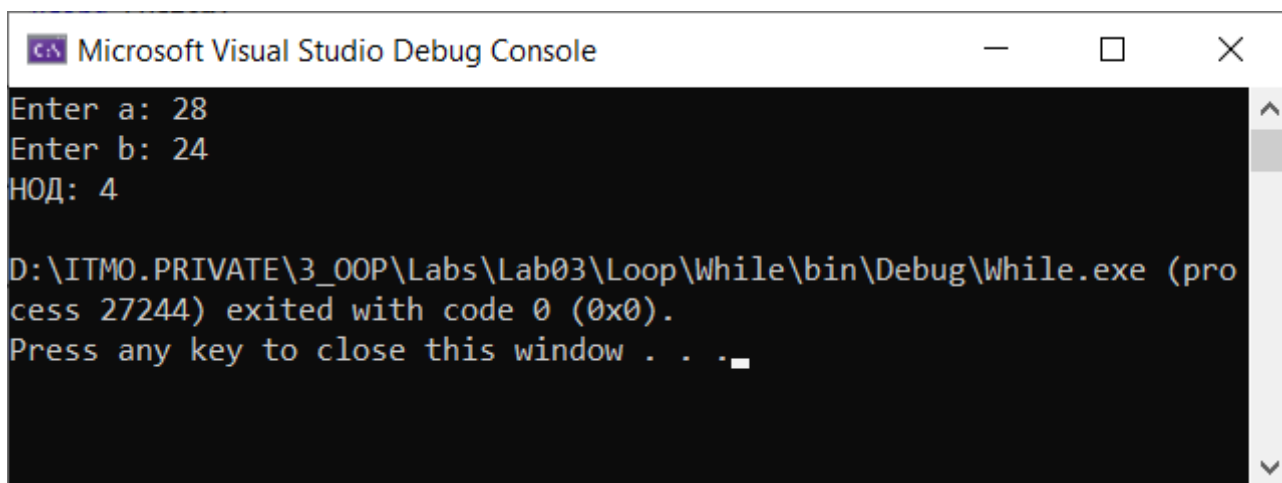
В проекте «DoWhile» была реализована (рисунок 9) и протестирована (рисунок 6) программа, выводящая таблицу синусов.



```
Microsoft Visual Studio Debug Console
Enter x1: 1
Enter x2: 1,2
x      | sin(x)
1,00   | 0,8415
1,01   | 0,8468
1,02   | 0,8521
1,03   | 0,8573
1,04   | 0,8624
1,05   | 0,8674
1,06   | 0,8724
1,07   | 0,8772
1,08   | 0,8820
1,09   | 0,8866
1,10   | 0,8912
1,11   | 0,8957
1,12   | 0,9001
```

Рисунок 6 — Вывод таблицы синусов на заданном интервале (тест)

В проекте «While» была реализована (рисунок 8) и протестирована (рисунок 7) программа, реализующая алгоритм Евклида.



```
Microsoft Visual Studio Debug Console
Enter a: 28
Enter b: 24
НОД: 4
D:\ITMO.PRIVATE\3_OOP\Labs\Lab03\Loop\While\bin\Debug\While.exe (process 27244) exited with code 0 (0x0).
Press any key to close this window . . .
```

Рисунок 7 — Алгоритм Евклида (тест)

```

using System;

namespace While
{
    0 references
    internal class While
    {
        0 references
        static void Main(string[] args)
        {
            int a, b, temp;

            Console.Write("Enter a: ");
            a = int.Parse(Console.ReadLine());
            Console.Write("Enter b: ");
            b = int.Parse(Console.ReadLine());

            temp = a;
            while (temp != b)
            {
                a = temp;
                if (a < b)
                {
                    temp = a;
                    a = b;
                    b = temp;
                }
                temp = a - b;
                a = b;
            }

            Console.WriteLine("НОД: {0}", a);
        }
    }
}

```

Рисунок 8 — Алгоритм Евклида (код)

```
using System;
```

```
namespace Loop
```

```
{
```

```
    0 references
```

```
    internal class DoWhile
```

```
    {
```

```
        0 references
```

```
        static void Main(string[] args)
```

```
        {
```

```
            double x, x1, x2, y;
```

```
            Console.Write("Enter x1: ");
```

```
            x1 = double.Parse(Console.ReadLine());
```

```
            Console.Write("Enter x2: ");
```

```
            x2 = double.Parse(Console.ReadLine());
```

```
            Console.WriteLine("x\t| sin(x)");
```

```
            x = x1;
```

```
            do
```

```
            {
```

```
                y = Math.Sin(x);
```

```
                Console.WriteLine("{0:f2}\t|{1:f4}", x, y);
```

```
                x += 0.01;
```

```
            }
```

```
            while (x <= x2);
```

```
        }
```

```
    }
```

```
}
```

Рисунок 9 — Вывод таблицы синусов на заданном интервале (код)

Цикл с предусловием целесообразнее выполнять если операции необходимы при определённом условии, цикл с постусловием необходим, когда необходимо выполнение операций хотя бы 1 раз.

2.2 Задание 2

В проекте «For» была реализована (рисунок 10) и протестирована (рисунок 11) программа, рассчитывающая сумму от 0 до k и от m до 100.

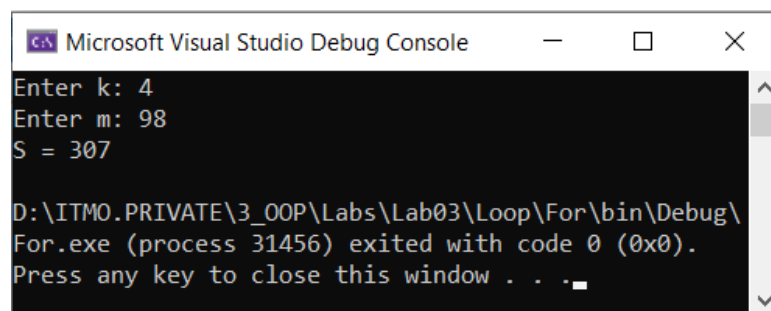
```
using System;

namespace For
{
    0 references
    internal class For
    {
        0 references
        static void Main(string[] args)
        {
            try
            {
                Console.Write("Enter k: ");
                int k = int.Parse(Console.ReadLine());
                if (k > 100 || k < 0)
                {
                    Console.WriteLine("Input must be an integer in range from 0 to 100 inc!");
                    return;
                }

                Console.Write("Enter m: ");
                int m = int.Parse(Console.ReadLine());
                if (m > 100 || m <= k)
                {
                    Console.WriteLine("Input must be an integer in range from {0} to 100 inc!", k);
                    return;
                }

                int s = 0;
                for (int i = 1; i <= 100; i++)
                {
                    if (i > k && i < m) continue;
                    s += i;
                }
                Console.WriteLine("S = {0}", s);
            }
            catch (FormatException)
            {
                Console.WriteLine("Input must be an integer in range from 0 to 100 inc!");
            }
        }
    }
}
```

Рисунок 10 — Расчёт суммы (код)



```
Microsoft Visual Studio Debug Console
Enter k: 4
Enter m: 98
S = 307

D:\ITMO.PRIVATE\3_OOP\Labs\Lab03\Loop\For\bin\Debug\
For.exe (process 31456) exited with code 0 (0x0).
Press any key to close this window . . .
```

Рисунок 11 — Расчёт суммы (тест)

2.3 Задание 3

Для выполнения задания был выбран 2 вариант (рисунок 12).

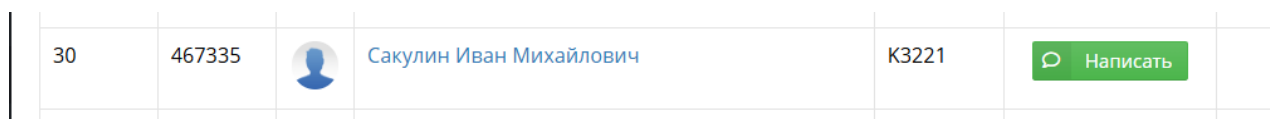


Рисунок 12 — Выбор 2 варианта

В проекте «Shooting» была реализована (рисунок 14) и протестирована (рисунок 13) программа.

Сначала пользователю предлагается ввести количество выстрелов (с проверкой ввода), затем в цикле запрашиваются координаты выстрелов, при ошибке ввода координаты спрашиваются снова.

Также дополнительно (*) добавлены сдвиг центра мишени (в пределах радиуса 3) и помехи (небольшой разброс).

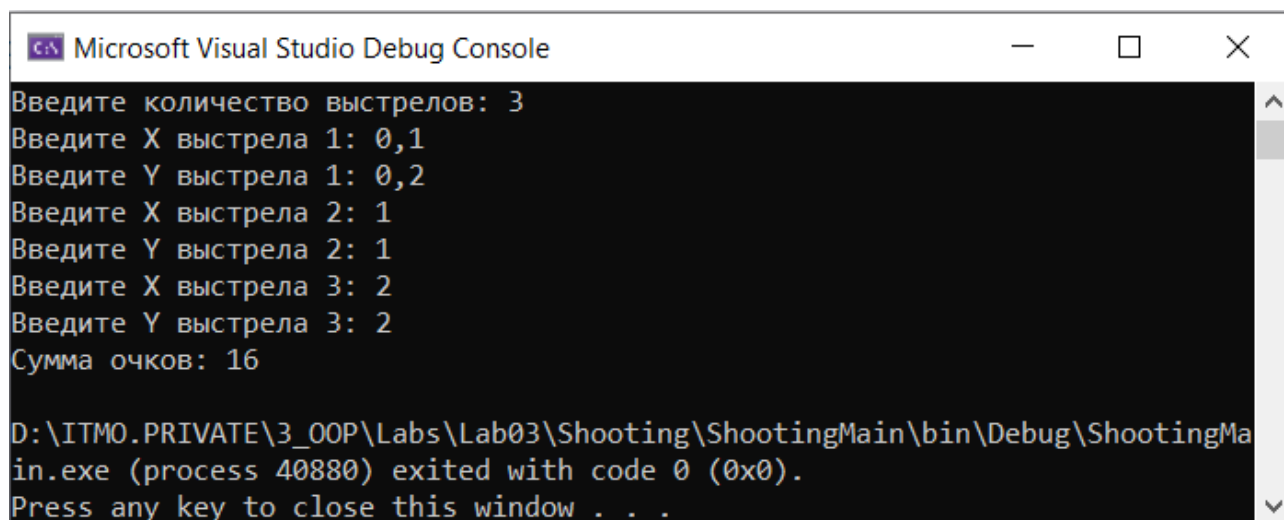


Рисунок 13 — Пример диалога с пользователем

```
1 using System;
2 namespace ShootingAddOn
3 {
4     internal class ShootingAddOn
5     {
6         static void Main(string[] args)
```

```

7      {
8          try
9          {
10             // Input shots count
11             Console.Write("Введите количество выстрелов: ");
12             int shotsCount = int.Parse(Console.ReadLine());
13             if (shotsCount <= 0)
14             {
15                 Console.WriteLine("Должно быть положительное число!");
16                 return;
17             }
18
19             // Randomize target *
20             Random rnd = new Random();
21             double targetX = (rnd.NextDouble() * 6) - 3;
22             double targetY = (rnd.NextDouble() * 6) - 3;
23
24             // Recording of shots
25             int totalScore = 0;
26             int i = 0;
27             while (i < shotsCount)
28             {
29                 try
30                 {
31                     // Input
32                     Console.Write("Введите X выстрела {0}: ", i + 1);
33                     double x = double.Parse(Console.ReadLine());
34                     Console.Write("Введите Y выстрела {0}: ", i + 1);
35                     double y = double.Parse(Console.ReadLine());
36
37                     // Transform to target coordinate system *
38                     x -= targetX;
39                     y -= targetY;
40
41                     // Noise *
42                     x += (rnd.NextDouble() * 0.1) - 0.05;
43                     y += (rnd.NextDouble() * 0.2) - 0.1;
44
45                     // Check

```

```

46         double radius = x * x + y * y;
47         if (radius <= 1)
48             totalScore += 10;
49         else if (radius <= 4)
50             totalScore += 5;
51         else if (radius <= 9)
52             totalScore += 1;
53
54         i++;
55     }
56     catch (FormatException e)
57     {
58         Console.WriteLine("Некорректная координата: {0}",
59             ↪ e.Message);
60     }
61     catch (OverflowException e)
62     {
63         Console.WriteLine("Слишком большая координата: {0}",
64             ↪ e.Message);
65     }
66
67     Console.WriteLine("Сумма очков: {0}", totalScore);
68 }
69 catch (FormatException e)
70 {
71     Console.WriteLine("Должно быть положительное целое число:
72     ↪ {0}", e.Message);
73 }
74 catch (OverflowException e)
75 {
76     Console.WriteLine("Слишком большое число: {0}", e.Message);
77 }
78 }

```

Рисунок 14 — C# код

2.4 Код-ревью

Резюме

Данный код реализует простую игру, в которой пользователь вводит количество выстрелов и координаты каждого выстрела, а программа вычисляет и выводит итоговый счет. Код содержит обработку исключений для некорректного ввода данных, что является положительным моментом.

Ошибка

Код в целом работает корректно, однако есть возможность улучшить обработку ошибок. Например, если пользователь введет нечисловое значение, программа завершится с ошибкой. Рекомендуется использовать `TryParse` для безопасного парсинга.

Стиль кода

Стиль кода в целом соответствует стандартам C#. Однако, для улучшения читаемости, можно использовать более описательные имена переменных и методов. Например, вместо `i` можно использовать `shotIndex`.

Структура кода

Структура кода логична, однако можно выделить отдельные методы для обработки ввода и вычисления счета. Это улучшит модульность и тестируемость кода.

Читаемость

Код читаем, но можно улучшить комментарии, чтобы они более четко объясняли логику. Например, комментарии к блокам кода могут быть более детализированными.

Производительность

Производительность кода на текущем уровне приемлема. Однако, если количество выстрелов будет очень большим, стоит рассмотреть возможность оптимизации, например, путем использования массивов для хранения координат.

Масштабируемость

Код не очень масштабируем, так как все логика находится в одном методе. Рекомендуется разбить код на несколько методов, чтобы упростить добавление новых функций в будущем.

Безопасность

Код не содержит явных уязвимостей, однако стоит учитывать возможность ввода некорректных данных. Использование TryParse поможет избежать потенциальных проблем.

Обработка ошибок

Обработка ошибок реализована, но можно улучшить, добавив более детализированные сообщения об ошибках и возможность повторного ввода данных в случае некорректного ввода.

Заключение

Код представляет собой хорошую основу для игры, однако требует доработки в плане структуры, читаемости и обработки ошибок. Рекомендуется выделить отдельные методы для улучшения модульности и тестируемости, а также использовать более безопасные методы ввода данных.

ЗАКЛЮЧЕНИЕ

В ходе работы были изучены управляющие операторы (if, else, switch, while, do-while, for) и написаны программы с их участием.